Evaluating the Ripple Effects of Knowledge Editing in Language Models

Roi Cohen[1] **Eden Biran**[1] **Ori Yoran**[1] **Amir Globerson**[1,2] **Mor Geva**[1,2,*]

[1]Blavatnik School of Computer Science, Tel Aviv University [2]Google Research

{roi1, edenbiran, oriy}@mail.tau.ac.il, {gamir, morgeva}@tauex.tau.ac.il

arXiv:2307.12976v2 [cs.CL] 20 Dec 2023

Abstract

Modern language models capture a large body of factual knowledge. However, some facts can be incorrectly induced or become obsolete over time, resulting in factually incorrect generations. This has led to the development of various editing methods that allow updating facts encoded by the model. Evaluation of these methods has primarily focused on testing whether an individual fact has been successfully injected, and if similar predictions for other subjects have not changed. Here we argue that such evaluation is limited, since injecting one fact (e.g. *"Jack Depp is the son of Johnny Depp"*) introduces a "ripple effect" in the form of additional facts that the model needs to update (e.g., *"Jack Depp is the sibling of Lily-Rose Depp"*). To address this, we propose novel evaluation criteria that consider the implications of an edit on related facts. Using these criteria, we then construct RIPPLEEDITS, a diagnostic benchmark of 5K factual edits, capturing various types of ripple effects. We evaluate prominent editing methods on RIPPLEEDITS, showing that they fail to introduce consistent changes in the model's knowledge. In addition, we find that a simple in-context editing baseline obtains the best scores on our benchmark, suggesting a promising research direction for model editing.[1]

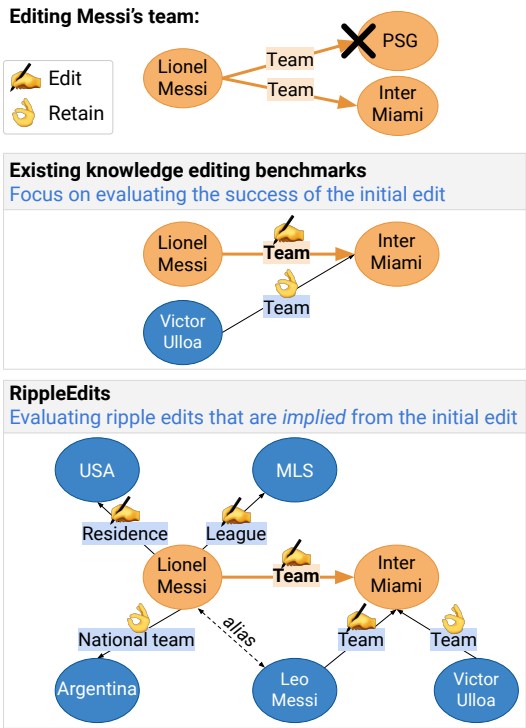

Figure 1: Illustration of the evaluation scope of RIPPLEEDITS, compared to existing knowledge editing benchmarks. For a given factual edit, we consider the "ripple effect" of the edit on the model's knowledge.

1 Introduction

Modern language models (LMs) capture a large volume of factual knowledge in their parameters, which can be effectively utilized in downstream tasks (Petroni et al., 2019; Roberts et al., 2020; Shin et al., 2020; Razniewski et al., 2021; Heinzerling and Inui, 2021; Kadavath et al., 2022; Cohen et al., 2023a). However, factual beliefs captured by the model may be incorrect or become outdated over time, potentially affecting the model's performance on downstream tasks, its reliability and its usability (Dhingra et al., 2022; Lazaridou et al., 2021; Jang et al., 2022).

This limitation has prompted research on knowledge editing (KE) methods, which modify LMs to fix their factual errors (we provide a formal definition in §2). Knowledge editing work has focused on applying factual updates to LMs. Given an entity-relation-object triplet (e, r, o) representing a fact (e.g. *"Lionel Messi plays for the Inter Miami team"*), recent work proposed various methods (Mitchell et al., 2022a; Meng et al., 2022, 2023; Hernandez et al., 2023b; Si et al., 2023) to inject

*Work done at Google DeepMind.

[1]We release RIPPLEEDITS and our code at https://github.com/edenbiran/RippleEdits.

this fact into the parameters of a given LM, while "overriding" beliefs the model might have on e and r (e.g. that Messi plays for Paris Saint-Germain).

A key question with KE is how to evaluate the success of such editing operations. The most basic "sanity-check" is that the model correctly completes $(e, r, ?)$, as well as other paraphrases of this task, with o. However, this is not enough as an evaluation, since one needs to check that the model did not distort other facts. Indeed, the standard evaluation protocol (Mitchell et al., 2022b; Meng et al., 2022, 2023) for KE focuses on these two aspects of correctly completing various paraphrases of the new fact, as well as ensuring that other unrelated facts have not been changed.

In this work, we argue that to evaluate model edits, one should go beyond the single fact that was edited and check that other facts that are logically derived from the edit were also changed accordingly. For example, if z is the mother of e, then the children of z are the siblings of e. Consequently, once we modify the belief of a certain model that $z \rightarrow z'$ is the mother of e, then we should also ensure that the model's belief regarding the siblings of e is also correct. Fig. 1 illustrates another example, where editing the `Team` for which `Lionel Messi` plays modifies other related facts, such as his country of residence, while other facts should be retained. We refer to such changes that are implied by a factual edit as *"ripple effects"*.

To account for ripple effects in the evaluation of factual edits, we propose six concrete evaluation criteria (see §3, Fig. 2), for testing which facts other than the edit itself should be modified or retained post-editing. Our tests evaluate how well the model integrates the edit with the rest of its knowledge, through queries that involve logical reasoning, complex composition of facts with the edit as an intermediate step, subject aliasing, and specificity across relations.

Building upon these criteria, we create RIPPLEEDITS, a new benchmark for comprehensive evaluation of KE of LMs (see §4). RIPPLEEDITS includes 5K entries, each consisting of a factual edit, along with a set of test queries that check if the edit was successful in terms of its ripple effect. Moreover, RIPPLEEDITS contains meta-data for each edit, including information about the timestamp of the edit (i.e., recent versus old), and the popularity of the entities (i.e., head versus tail).

We use RIPPLEEDITS to evaluate three popular editing methods on five recent strong LMs (see §5). We find that, even though current KE methods are effective in modifying a particular fact, they often fail to capture the ripple effects entailed by that fact, and demonstrate poor performance on most of our evaluation criteria. Moreover, analyzing how editing performance varies across model sizes and entity frequencies, we find that (a) larger models handle ripple effects better, and (b) editing frequent entities results in more logical reasoning errors.

Last, we consider a simple in-context editing baseline for KE, that leverages the casual attention mechanism rather than explicit parametric updates. While this method achieves the best results on our benchmark, outperforming current parametric KE methods, there is still ample room for improvement that calls for future research.

To conclude, our work makes multiple contributions: (a) it highlights key limitations of KE evaluation, specifically regarding ripple effects and introduces comprehensive evaluation criteria to mitigate those limitations, (b) it proposes RIPPLEEDITS, a benchmark inspired by these criteria, (c) it evaluates current methods for KE and shows that they do not perform well on this task, while demonstrating that in-context editing is a promising direction for KE. We release RIPPLEEDITS and our code to facilitate future work on KE.

2 Problem Setting

We consider editing of *factual knowledge*, where facts are expressed as triplets (e, r, o) of a subject entity e (e.g. `Lionel Messi`), a relation r (e.g. `Team`), and an object o (e.g. `Inter Miami`). We distinguish between two edit types, based on the knowledge encoded in the model before the edit: (a) *modification* of a fact that is already encoded in the model $(e, r, o) \rightarrow (e, r, o^*)$, that is, updating the object $o \rightarrow o^*$ for a given subject e and relation r, and (b) *injection* of a new fact (e, r, o^*) that is not captured by the model. Moreover, we note that for one-to-one relations like `Date of birth`, where there is a single object for a given subject, an injection edit can be viewed as populating an empty object $(e, r, \emptyset) \rightarrow (e, r, o^*)$. In contrast, for one-to-many relations, such as `Sibling` and `Occupation`, an injection edit augments the set of objects $(e, r, \{o_1, .., o_n\}) \rightarrow (e, r, \{o_1, .., o_n, o^*\})$. Whether an edit is viewed as a modification or injection, depends on whether that information was captured in the model before the edit. Moreover,

evaluating if a specific fact (before or after an edit) is encoded by a model is typically done by testing if the model predicts the object for various input queries that represent the subject and relation (see more details in §3.2).

3 Ripple Effects of Factual Edits

We focus on evaluating the downstream effect of a given edit, i.e., given an edit $(e, r, o) \rightarrow (e, r, o')$, we expect certain facts related to the edit to change as well. Consider, for example, the edit shown in Fig. 1. Changing the team for which Messi plays might also affect the league he plays in and his country of residence. Formally, for a given model, assume a knowledge-graph $\mathcal{K} := \{(e_i, r_i, o_i)\}_{i=1}^{N}$ of N factual triplets, representing the model's knowledge, and let $\delta : (e, r, o) \rightarrow (e, r, o')$ be an edit request for $\mathcal{K}$. We define the *ripple effect* of δ on $\mathcal{K}$, as the set of triplets $\mathcal{R}(\delta)$ that the model implicitly needs to inject, modify, or delete from $\mathcal{K}$ to reflect the world state after the edit.

Notably, different edits can cause ripple effects of varying magnitudes. For example, changing the country of Rome from Italy to France, will entail many follow-up changes, such as the country in which the Colosseum is located, the language spoken in Rome, and so forth. In contrast, updating the siblings of Prince (Fig. 2) is both more realistic and should result in a more local effect. We refer to the number of facts affected by a single edit δ (i.e. $|\mathcal{R}(\delta)|$) as its *severity*. In general, editing popular entities that appeared frequently during training is likely to introduce more changes, and thus, editing their properties has a higher severity.

3.1 Evaluation Criteria

We wish to evaluate how well models capture the ripple effects of factual edits. However, given that ripple effects can potentially span a large number of implied edits, we focus on evaluating modified facts that are within a 2-hop distance from the subject or object of the edit. Concretely, for an edit $\delta : (e, r, o) \rightarrow (e, r, o^*)$, we evaluate the ripple effect $\mathcal{R}(\delta)$, via the following evaluation criteria (examples are shown in Fig. 2):

1. ***Logical Generalization*** **(LG)**: Relations in a knowledge graph satisfy certain logical constraints. For example, the relation `Sibling` is symmetric and therefore if $(e, \texttt{Sibling}, o)$ is true then $(o, \texttt{Sibling}, e)$ is also true, and vice versa (Fig. 2A). Likewise, the relation `Location` is transitive so $(e, \texttt{Location}, o) \wedge (o, \texttt{Location}, z) \Rightarrow (e, \texttt{Location}, z)$. We wish to check that such logical implications about the subject e, the original object o, and the new object o^*, hold after editing. We focus and elaborate on specific constraints in §4.
2. ***Compositionality I*** **(CI)**: As δ alters one edge in a knowledge graph, we can check the composition of this edge with other edges. Namely, we test if the model can compose the edited fact with other facts about the target object. Let (o, r', z) and (o^*, r', z^*) be two facts of the same relation about o and o^*, respectively. Also, denote by $r'' = r \circ r'$ the complex relation expressing the composition of r and r' (e.g., r'' = `Profession of sibling` for r = `Sibling` and r' = `Profession`). Then, after the edit δ, we expect the following change $(e, r'', z) \rightarrow (e, r'', z^*)$. For example (Fig. 2B), the professions of the siblings of `Prince` can be modified once a new sibling is injected.
3. ***Compositionality II*** **(CII)**: We test if the model can compose the edited fact with facts about a different subject $e' \neq e$. Formally, let (e', r', e) be a fact about e' with e as its object, and denote by $r'' = r' \circ r$ the complex relation expressing the composition of r' and r (see an example in criterion 2). After the edit δ, the following change is expected for the subject e': $(e', r'', o) \rightarrow (e', r'', o^*)$. For instance (Fig. 2C), changing the siblings of `Prince` also modifies the siblings of the founder of `Paisley Park Records` (i.e., r'' is a complex relation expressing "siblings of the founder").
4. ***Subject Aliasing*** **(SA)**: We test that editing a fact about e induces the same edit to other entities e' that are aliases of e, namely, $(e', r, o) \rightarrow (e', r, o^*)$. For instance (Fig. 2D), modifying the siblings of `Prince`, should also modify the sibling of his alias, `Prince Roger Nelson`.
5. ***Preservation*** **(PV)**: If r is a one-to-many relation, then adding a new object should not affect the other objects encoded about e. Hence, in such cases, we expect that any existing triplet (e, r, o') for an object $o' \neq o^*$ would remain following the edit. For example (Fig. 2E), after inserting the sibling `Nicholas Carminowe` for `Prince`, the fact that `Tyka Nelson` is also his sibling should be retained.
6. ***Relation Specificity*** **(RS)**: We test that facts